

Git / GitHub 가이드 (한 사이클)

Feature Branch Workflow · CAB302 Shapes OOP · [Atlassian](#) · [각주](#)

1→9가 한 사이클. **최초** = 처음·새 PC·새 계정·새 저장소일 때만 (끝났으면 건너뛰기). 표시 없음 = 기능 추가할 때마다.

[최초] 계정·Git·저장소 → config → clone
① main 맞춤 → ② feature → ③ commit → ④ push → ⑤ PR → ⑥ 피드백 → ⑦ merge → ⑧ 정리
→ 다음 기능은 ②(실제로는 단계 2)부터

1. 시작 준비 **최초** 중심
2. main 맞춤
3. feature
4. add / commit
5. push
6. PR · 리뷰
7. 피드백
8. merge
9. 정리

1. 시작 준비 — 계정 · Git · 저장소

대부분 **최초**. URL 있고 clone까지 끝났으면 [2번](#)으로.

1-a. GitHub 계정 **최초**

어디서	할 일
github.com → Sign up	가입 · 이메일 인증
프로필 → Settings → Emails	커밋용 이메일 ^[1]
(선택) Profile	이름·사진

1-b. Git 설치 **최초**

Syntax	의미	사용 예
git-scm.com 설치	Windows에 Git	기본 옵션 OK
git --version	설치 확인	git version 2.x.x

IntelliJ: Settings → Version Control → Git

옵션	의미
GitHub Desktop	GUI로 clone-commit-push

1-c. 저장소 확보 **최초** · clone보다 앞 ^[2]

① 수업/팀 저장소 있음

#	GitHub	의미
1	원본 페이지	수업 org 등
2	Fork	내 복사본
3	또는 Settings → Collaborators	초대 수락

② 새로 만든다

#	GitHub	의미
1-2	+ → <i>New repository</i> · 이름 · Public/Private	
3	"Add a README" 권장 [3]	첫 커밋
4-5	<i>Create</i> · <i>Collaborators</i> 초대	

1-d. 로컬 신원 **최초** · PC/계정

Syntax	의미	사용 예
git config --global user.name "..."	커밋 이름 [1]	..."Alice Kim"
git config --global user.email "..."	Emails와 동일	
git config --global -l	확인 (보조)	

옵션 / 보조	의미	사용 예
git config --global color.ui auto	터미널 색 자동	
git config --global core.excludesfile <파일>	PC 전역 ignore	[19]

1-e. 로컬에 저장소 두기 **최초** — 보통 clone / 옵션 init

권장 (원격이 이미 있을 때): git clone

Syntax	의미	사용 예
Code → URL 복사	HTTPS	
git clone <URL>	원격→로컬 [4]	git clone https://...
cd <폴더>	이동	

이미 clone됐으면 재clone 말고 **2번** 로그인/토큰 [5]

옵션 (로컬에서 먼저): git init — Canvas식. clone과 둘 다 하지 않음.

Syntax	의미	사용 예
cd · mkdir · cd	프로젝트 폴더	mkdir example-repo
(선택) mkdir src	소스 폴더	IntelliJ식
git init	빈 저장소(.git) [4]	
파일 → add → commit	첫 로컬 커밋	4번과 동일
GitHub 빈 저장소 (README 없이)	1-c 경우 ②	
git remote add origin <URL>	원격 연결	
git push -u origin main	첫 업로드	

보조	의미	
git remote -v	origin 확인	
git remote add upstream <URL>	Fork 원본 [6]	최초 Fork
SSH keys + ssh-keygen	SSH [7]	최초 선택

1-f. main 보호 **최초** · 팀장 권장

Settings → Branches / Rules → main 직접 push 금지 [8]

1-g. .gitignore **최초** · 옵션

커밋하면 안 되는 패턴을 루트 .gitignore에 작성 [19]

예	의미
logs/ · *.class · .idea/	로그·빌드·IDE
git add .gitignore → commit	규칙도 버전 관리

1-h. 확인 최초 직후 · 또는 의심 시

확인	기대
git status · git log --oneline -5	main · 커밋 보임
GitHub 페이지 · IntelliJ Open	로컬과 대응

2. 최신 main 맞추기 [매 사이클]

Syntax	의미	사용 예
git checkout main / switch main	공식 브랜치	
git pull origin main	확정본 동기화 ^[9]	

브랜치 main · Fork면 Sync fork 후 pull

보조	의미	주의
git fetch origin	원격 정보만	병합 안 함
git merge origin/main	fetch 후 병합	pull과 유사
git reset --hard origin/main	강제 일치 ^[10]	미저장 삭제
git reset --hard <커밋>	지정 커밋으로 맞춤	공유 main 비권장

3. Feature branch [매 사이클]

Syntax	의미	사용 예
git checkout -b <이름>	새 갈래 ^[11]	... feature/add-square

(선택) 웹 Create branch · 보통 로컬 후 5번 push

보조 / 옵션	의미	사용 예
git branch / -a	로컬 / 원격 목록	
git branch <이름>	생성만 (이동 안 함)	이후 checkout
git checkout <이름>	기존 브랜치 이동	

4. 수정 → Stage → Commit [매 사이클]

Syntax	의미	사용 예
(에디터 수정)	코드	Square.java
git status	변경 확인 ^[12]	
git add <파일>	stage	git add src/shapes/Square.java
git commit -m "..."	로컬 기록 ^[13]	..."Add Square class"

IntelliJ: 체크= add, Commit= commit

보조 — stage/차이/로그	의미
git add . · git diff · git diff --staged	전체 stage / 미·스테이지 차이
git restore --staged <파일> · git reset <파일>	stage 취소 (내용 유지)
git log · --oneline · --follow <파일> · --stat -M	히스토리 / 리네임
git log B..A · git diff B...A · git show <SHA>	브랜치 비교 / 커밋 내용

보조 — 삭제·이동	의미
<code>git rm <파일> · git mv <옛> <새></code>	삭제/이동 + stage
보조 — stash	의미
<code>git stash · stash list · stash pop · stash drop</code>	임시 저장 / 목록 / 적용 / 버리기

5. Feature Push [매 사이클]

Syntax	의미	사용 예
<code>git push -u origin <브랜치></code>	원격 업로드 ^[14]	<code>... feature/add-square</code>
<code>git push</code>	(-u 이후)	

*Compare & pull request · Branches · main*은 아직 그대로

6. Pull Request · 리뷰 [매 사이클]

PR = GitHub 기능 ^[15]

#	GitHub	의미
1	<i>Pull requests → New</i>	요청
2	<code>base= main ← compare= feature/...</code>	방향
3-4	제목 · Reviewers · Create	
5-6	<i>Files changed / Review · Conversation</i>	Approve 등

선택	의미
<code>gh pr create</code>	CLI PR
<i>Checks / Actions</i>	CI 있으면 통과 확인

7. 피드백 반영 [필요 시]

Syntax	의미
<code>수정 → add → commit → push</code>	같은 PR에 반영

Conversation · Reply / Resolve

보조 (충돌)	의미
<code>main pull 후 feature에서 git merge main</code>	최신 반영 ^[16]
<code><<<<<<< 정리 → add → commit → push</code>	충돌 해결
<code>PR Resolve conflicts</code>	웹 해결
<code>git rebase main</code>	재배치 ^[17] . 익숙해진 뒤

8. Merge [매 사이클]

GitHub	의미
<i>Merge pull request → Confirm</i>	<code>feature→main</code> ^[18]
(선택) <i>Delete branch</i>	원격 feature 삭제

```
git checkout main
git pull
git pull origin feature/add-square
git push
```

위 = 웹 대신 로컬 merge (보조)

보조	의미
<code>git merge <브랜치></code>	지정 브랜치를 현재에 병합
<code>git revert <커밋></code>	main 실수 취소

9. 정리 · 다음 사이클 [매 사이클]

Syntax	의미	사용 예
<code>git checkout main</code>	공식 복귀	
<code>git pull origin main</code>	최신 main	
<code>git branch -d <feature></code>	로컬 feature 삭제	... feature/add-square

Delete branch · (선택) Releases / tag v1.0

다음 기능 → [2번](#)부터. 새 PC/새 저장소 → [1번](#)부터. 저장소 생성 시점 → 1-c (clone 1-e보다 앞).

각주

- 커밋 메타데이터. GitHub Emails에 같은 주소 필요. `--global` =이 PC 사용자 전체. [↑ 본문으로](#) [↑](#)
- 저장소 없으면 clone URL 없음. 과제는 Fork/Collaborator가 흔함. [↑ 본문으로](#)
- README 시 원격 첫 커밋. 빈 저장소면 로컬 첫 커밋 후 push. [↑ 본문으로](#)
- `clone` =원격 복사+히스토리+origin. `init` =현재 폴더에 빈 저장소만(원격은 나중에 `remote add + push`). 같은 폴더에 둘 다 쓰지 않음. [↑ 본문으로](#)
- HTTPS는 Token/Credential/SSH. IntelliJ 브라우저 로그인 가능. [↑ 본문으로](#)
- origin=내 fork, upstream=원본. `fetch upstream` → `merge` → `push origin main`. [↑ 본문으로](#)
- SSH는 나중에 가능. 공개키만 GitHub. [↑ 본문으로](#)
- main 실수 push 거절 → feature→PR만. [↑ 본문으로](#)
- `pull` ≈`fetch`+`merge`. 최신 main에서 feature 분기. [↑ 본문으로](#)
- `reset --hard`는 로컬 변경 삭제 가능. [↑ 본문으로](#)
- 예: `feature/add-square`. “main만 공식”은 팀 규칙. [↑ 본문으로](#)
- Untracked→Staged→Unmodified. Stage=이번 커밋만 고르기. [↑ 본문으로](#)
- Commit=로컬. GitHub는 push. [↑ 본문으로](#)
- `-u` 이후 `git push` 만. feature push≠main 변경. [↑ 본문으로](#)
- PR=병합 요청+리뷰. 중간 도움 요청에도. [↑ 본문으로](#)
- 충돌=같은 줄 동시 수정. 자주 2번으로 예방. [↑ 본문으로](#)
- Rebase=히스토리 재작성. 초심자는 merge. [↑ 본문으로](#)
- Merge 후 main=새 확정본. Squash/Merge 팀 통일. [↑ 본문으로](#)
- `.gitignore` =저장소별 무시. `core.excludesfile` =PC 전역. 이미 추적 중이면 ignore만으로 안 빠짐 → `git rm --cached`. [↑ 본문으로](#)